step2_all_data_processing_refactored

February 6, 2026

[123]:
```
import pandas as pd
import numpy as np
import os
import re
```

[124]:
```
# ========================================
# 1.
# ========================================
DATA_DIR = '../../data'
INPUT_DIR = f'{DATA_DIR}/step1_dwh_outputs'
OUTPUT_DIR = f'{DATA_DIR}/step2_processing_outputs'

#  :
FILE_NAMES = {
    '  ': 'step1_all_sample_rslt.parquet',
    '  ': 'step1_all_injection.parquet',
    '  ': 'step1_all_drug_taken.parquet',
    '  ': 'step1_all_vital.parquet',
    '  ': 'step1_all_somatometry.parquet',
    '  ': 'step1_all_cancer.parquet',
}

#
GCSF_CODES = ["3399405", "3399406", "3399408"]  #
GLAS_CODES = ["3399410"]                        #

REQ_EXAM_ITEMS = {'  NEUT#', '  WBC'}
CANCER_EXCLUDE_PATTERN = r'^C(42|8[1-9]|9[0-6])$'

#
TH_NEUT_SEVERE = 0.5
TH_NEUT_MODERATE = 1.0
TH_FEVER = 38.5

#    (Day 1  )
WINDOW_EVENT_END_DAY = 28
WINDOW_PRE_GCSF_DAYS = 3
```

[125]:
```
# ========================================
# 2.
# ========================================
def load_integrated_data(input_dir, file_mapping):
    """
      Parquet
    """
    dfs = {}
    print(f"     : {input_dir}")

    for key, file_name in file_mapping.items():
        file_path = os.path.join(input_dir, file_name)

        if os.path.exists(file_path):
            try:
                #
                df = pd.read_parquet(file_path)
                dfs[key] = df
                print(f"    : {key} (Records: {len(df)})")
            except Exception as e:
                print(f"  : {file_path}     : {e}")
                dfs[key] = pd.DataFrame()
        else:
            #    .parquet
            file_path_with_ext = file_path + '.parquet'
            if os.path.exists(file_path_with_ext):
                try:
                    df = pd.read_parquet(file_path_with_ext)
                    dfs[key] = df
                    print(f"    (.parquet ): {key} (Records: {len(df)})")
                except Exception as e:
                    print(f"  : {file_path_with_ext}     : {e}")
                    dfs[key] = pd.DataFrame()
            else:
                print(f" :        : {file_path}")
                dfs[key] = pd.DataFrame()

    return dfs

def get_latest_record_before_target(df_target_dates, df_source,␣
↪date_col_source, target_date_col='DO_DATE', on='hs'):
    """  (DO_DATE)        """
    merged = pd.merge(df_target_dates[[on, target_date_col]].drop_duplicates(),
                      df_source, on=on)

    filtered = merged[merged[date_col_source] <= merged[target_date_col]].copy()
```

```
    filtered['MAX_DATE'] = filtered.groupby([on,␣
↪target_date_col])[date_col_source].transform('max')

    return filtered[filtered[date_col_source] == filtered['MAX_DATE']].
↪drop(columns=['MAX_DATE'])
```

[133]:
```
# ========================================
# 3.
# ========================================
def process_drugs(dfs):
    """    """
    def _clean_drug(df):
        df = df.copy()
        df['DRUG_CD'] = df['DRUG_PRICE_LIST_CD'].str.slice(0, 7)
        return df.rename(columns={'QUERY_DATE': 'DO_DATE'})

    df_inje = _clean_drug(dfs['  '])
    df_drug = _clean_drug(dfs['  '])

    #  G-CSF (   )
    df_gcsf_therapeutic = df_inje.loc[df_inje['DRUG_CD'].isin(GCSF_CODES),␣
↪['hs', 'DO_DATE', 'DRUG_CD', 'EXEC_DOSE']].copy()
    df_gcsf_therapeutic = df_gcsf_therapeutic.rename(columns={'DO_DATE':␣
↪'GCSF_DATE'})

    #  G-CSF ( )
    df_glas_prophylactic = df_inje.loc[df_inje['DRUG_CD'].isin(GLAS_CODES),␣
↪['hs', 'DO_DATE', 'DRUG_CD', 'EXEC_DOSE']].copy()
    df_glas_prophylactic = df_glas_prophylactic.rename(columns={'DO_DATE':␣
↪'GLAS_DATE'})

    #
    df_chemo = pd.concat([df_drug, df_inje])
    df_chemo = df_chemo[df_chemo["ANTI_CANCER_FLG"] == " "].copy()

    return df_chemo, df_gcsf_therapeutic, df_glas_prophylactic

def filter_cancer_patients(df_canc):
    """          """
    df = df_canc.copy()
    df['DIAGNOSIS_DATE_2'] = pd.to_datetime(df['DIAGNOSIS_DATE_2'],␣
↪errors='coerce')
    df['DIAGNOSIS_CD_SHORT'] = df['DIAGNOSIS_CD'].str.slice(0, 3)

    min_dates = df.groupby('hs')['DIAGNOSIS_DATE_2'].transform('min')
    earliest_records = df[df['DIAGNOSIS_DATE_2'] == min_dates]
```

```
    exclude_mask = earliest_records['DIAGNOSIS_CD_SHORT'].str.
 ↪match(CANCER_EXCLUDE_PATTERN, na=False)
    hs_to_drop = earliest_records.loc[exclude_mask, 'hs'].unique()

    df_filtered = df[~df['hs'].isin(hs_to_drop)].copy()
    df_dummies = pd.get_dummies(df_filtered[['hs', 'DIAGNOSIS_CD_SHORT']],
                                columns=['DIAGNOSIS_CD_SHORT'],␣
 ↪prefix='DIAGNOSIS')
    return df_dummies.groupby('hs').sum().reset_index()

def create_event_labels(df_chemo, df_exam, df_vt, df_gcsf_therapeutic):
    """
      (Event)
    """
    # 1.

    df_exam_min = (df_exam[df_exam['EXAM_ITEM'].isin(REQ_EXAM_ITEMS)]
                   .groupby(['hs', 'QUERY_DATE', 'EXAM_ITEM'])['RESULT_NUM'].
 ↪min()
                   .unstack().reset_index()
                   .rename(columns={'  NEUT#': 'minNEUT#', '  WBC': 'minWBC'}))

    df_vt_max = (df_vt.groupby(['hs', 'QUERY_DATE'])['TEMPR'].max()
                 .reset_index(name='maxBT'))

    df_daily = pd.merge(df_exam_min, df_vt_max, on=['hs', 'QUERY_DATE'],␣
 ↪how='outer')

    # 2. FN
    condition_severe = df_daily['minNEUT#'] < TH_NEUT_SEVERE
    condition_mod_fever = (df_daily['minNEUT#'] < TH_NEUT_MODERATE) &␣
 ↪(df_daily['maxBT'] >= TH_FEVER)
    df_daily['FN_Event'] = np.where(condition_severe | condition_mod_fever, 1,␣
 ↪0)

    # 3.      (Day 1   Day 28)
    df_base = df_chemo[['hs', 'DO_DATE']].drop_duplicates()
    df_merged = pd.merge(df_base, df_daily, on='hs', how='inner')

    #
    days_diff_raw = (df_merged['QUERY_DATE'] - df_merged['DO_DATE']).dt.days

    #  Day 1    ( =0   -> Day 1)
    df_merged['Day_Num'] = days_diff_raw + 1
```

```
    #  : Day 1 <= Day <= 28
    in_window = (df_merged['Day_Num'] >= 1) & (df_merged['Day_Num'] <=␣
↪WINDOW_EVENT_END_DAY)
    df_merged['Valid_FN'] = np.where(in_window, df_merged['FN_Event'], 0)

    df_fn_res = df_merged.groupby(['hs', 'DO_DATE'])['Valid_FN'].sum().
↪reset_index()

    # 4.   G-CSF    (Day 1   Day 28)
    df_gcsf_merged = pd.merge(df_base, df_gcsf_therapeutic, on='hs', how='left')

    gcsf_diff_raw = (df_gcsf_merged['GCSF_DATE'] - df_gcsf_merged['DO_DATE']).
↪dt.days
    df_gcsf_merged['Day_Num'] = gcsf_diff_raw + 1

    #
    is_gcsf_event = (
        (df_gcsf_merged['Day_Num'] >= 1) &
        (df_gcsf_merged['Day_Num'] <= WINDOW_EVENT_END_DAY) &
        (df_gcsf_merged['EXEC_DOSE'] > 0)
    )
    df_gcsf_merged['Therapeutic_GCSF_Event'] = np.where(is_gcsf_event, 1, 0)
    df_gcsf_res = df_gcsf_merged.groupby(['hs',␣
↪'DO_DATE'])['Therapeutic_GCSF_Event'].sum().reset_index()

    # 5.
    df_final = pd.merge(df_fn_res, df_gcsf_res, on=['hs', 'DO_DATE'],␣
↪how='left')
    df_final['Event'] = ((df_final['Valid_FN'] +␣
↪df_final['Therapeutic_GCSF_Event'].fillna(0)) >= 1).astype(int)

    return df_final[['hs', 'DO_DATE', 'Event']]

def create_features(df_base, dfs, df_chemo, df_glas_prophylactic):
    """

    """
    # 1.    ( )
    df_exam_last = get_latest_record_before_target(
        df_base,
        dfs['  '][dfs['  ']['EXAM_ITEM'].isin(REQ_EXAM_ITEMS)],
        'QUERY_DATE'
    )
    df_feat_exam = (df_exam_last.groupby(['hs', 'DO_DATE',␣
↪'EXAM_ITEM'])['RESULT_NUM'].min()
```

```
                    .unstack().rename(columns={'  NEUT#': 'lastNEUT#', '  WBC':␣
↪'lastWBC'}).reset_index())
    df_feat_exam = pd.merge(df_feat_exam, df_chemo[['hs', 'DO_DATE', 'AGE']].
↪drop_duplicates(), on=['hs', 'DO_DATE'])

    # 2.
    df_soma = dfs['  '].sort_values(['hs', 'SOMA_DATE']).copy()
    for col in ['MS_HEIGHT', 'MS_WEIGHT', 'MS_BSA']:
        df_soma[col] = df_soma.groupby('hs')[col].ffill()

    df_soma_last = get_latest_record_before_target(df_base, df_soma,␣
↪'SOMA_DATE')
    df_feat_soma = df_soma_last.groupby(['hs', 'DO_DATE'])[['MS_HEIGHT',␣
↪'MS_WEIGHT']].median().reset_index()

    # 3.
    df_vt_last = get_latest_record_before_target(df_base, dfs['  '],␣
↪'QUERY_DATE')

    vital_cols = [
    "TEMPR",        #
    "PULSE",        #
    "RESP",         #
    "BPH",          #   High/
    "BPL"           #   Low/
    ]

    # 2.
    target_cols = [col for col in vital_cols if col in df_vt_last.columns]

    # 3. hs(ID) DO_DATE( )
    df_vt_feature = df_vt_last.groupby(['hs', 'DO_DATE'])[target_cols].median().
↪reset_index()

    # 4.
    df_feat_drug = (df_chemo.groupby(['hs', 'DO_DATE', 'DRUG_CD'])['EXEC_DOSE'].
↪sum()
                    .unstack(fill_value=0).reset_index())

    # 5.   G-CSF   (Day 1       )
    df_glas_pre = pd.merge(df_base, df_glas_prophylactic, on='hs', how='left')

    days_diff_raw = (df_glas_pre['GLAS_DATE'] - df_glas_pre['DO_DATE']).dt.days
    df_glas_pre['Day_Num'] = days_diff_raw + 1

    #  Day 1  (Day 0 )      (-9 )
```

```
    #  -3 <= day < 1
    in_range = (
        (df_glas_pre['Day_Num'] >= (1 - WINDOW_PRE_GCSF_DAYS + 1)) &
        (df_glas_pre['Day_Num'] < 1)
    )
    df_glas_pre.loc[~in_range, 'EXEC_DOSE'] = 0

    df_feat_glas = df_glas_pre.groupby(['hs', 'DO_DATE'])['EXEC_DOSE'].sum().
↪reset_index(name='TOTAL_GLAS_PREDOSE')

    # 6.
    df_sex = dfs['  '][['hs', 'PT_SEX']].drop_duplicates()
    df_sex['PT_SEX'] = df_sex['PT_SEX'].map({'M': 0, 'F': 1})

    return {
        'exam': df_feat_exam, 'soma': df_feat_soma, 'vt': df_vt_feature,
        'drug': df_feat_drug, 'glas': df_feat_glas, 'sex': df_sex
    }
```

[134]:
```
# ========================================
# 4.
# ========================================
def main():
    #
    dfs = load_integrated_data(INPUT_DIR, FILE_NAMES)

    if not dfs or dfs['  '].empty:
        print("       (  )       ")
        return

    df_chemo, df_gcsf_therapeutic, df_glas_prophylactic = process_drugs(dfs)
    df_cancer_feat = filter_cancer_patients(dfs['   '])

    #
    if df_chemo.empty:
        print("              ")
        return

    df_labels = create_event_labels(df_chemo, dfs['  '], dfs['   '],␣
↪df_gcsf_therapeutic)

    df_base = df_labels[['hs', 'DO_DATE']].drop_duplicates()
    features = create_features(df_base, dfs, df_chemo, df_glas_prophylactic)

    df_final = df_labels
    for name, df_feat in features.items():
```

```
        join_keys = ['hs', 'DO_DATE'] if 'DO_DATE' in df_feat.columns else␣
↪['hs']
        if not df_feat.empty:
            df_final = pd.merge(df_final, df_feat, on=join_keys, how='inner')

    if not df_cancer_feat.empty:
        df_final = pd.merge(df_final, df_cancer_feat, on='hs', how='inner')

    print(f"Output shape: {df_final.shape}")
    os.makedirs(OUTPUT_DIR, exist_ok=True)


    output_filename = 'step2_all_tagt_feat'

    df_final.to_parquet(f'{OUTPUT_DIR}/{output_filename}')
    df_final.to_excel(f'{OUTPUT_DIR}/{output_filename}_to_excel.xlsx',␣
↪sheet_name='tagt_feat')
    print("Processing complete.")

if __name__ == "__main__":
    main()
```

```
    : ../../data/step1_dwh_outputs
   :    (Records: 65977019)
   :    (Records: 3913939)
   :    (Records: 1364433)
   :    (Records: 8240502)
   :    (Records: 1450524)
   :    (Records: 15131)
Output shape: (59693, 292)
Processing complete.
```

[ ]: